Analysis: Saturnalia

Saturnalia: Analysis

The first problem in the first instance of this new Code Jam contest has a very straightforward solution:

- Determine the length of the given string; call it L.
- Output a line consisting of one + character, L+2 − characters, and one more + character.
- Output a line consisting of one | character, then one space, then the given string, then another space, then one more | character.
- Output another line identical to the first line described above.

It is important to avoid reading the input in a way that strips space characters, since those are considered part of the given string. Sample Case #4 calls attention to this issue.

Analysis: Seating Chart

Seating Chart: Analysis

Before assigning any particular people to particular tables, we can figure out how many people must sit at each table: M = **N** mod **K** of the tables will be "more full" and will have ceil(**N** / **K**) people each, and the other L tables will be "less full" and will have floor(**N** / **K**) people each.

Small dataset

In the Small dataset, there are at most 8 people, and at most 8 tables. These numbers are small enough that we can try a brute force strategy; here is one example. Number the people from 1 to **N**, and generate each of the **N**! permutations of the list [1, ..., **N**]. Then, for each permutation, proceed left to right through the permutation, filling up the first table in clockwise order with the appropriate number of people, then the second table, and so on. Turn each of these assignments into a sorted list of pairs of people who are sitting next to each other, with the numbers in each pair themselves sorted in ascending order. Maintain a set of all such lists found, discarding any duplicates, and then return the size of that set.

Large dataset

In the Large dataset, **N** can be as large as 20, so we need to use combinatorics. As in the Small dataset, the tricky part is to avoid counting the same arrangement more than once.

First, let's consider how we assign people to tables. If we have P people who are not yet assigned, and the table requires Q people, there are (P choose Q) ways of selecting those people.

Then, how many ways are there to put those Q people around the table? First, let's dispense with some small cases: for 1 or 2 people, there is only one way. Otherwise, without loss of generality, let's put the first person in our chosen set at the "top" of the table. Then, for each of the (Q-1)! possible orders of the remaining people, we can try putting them clockwise around the table, starting from the spot that is directly clockwise from our "top" person. Notice that for any one of these orders, using the reverse of that order creates an equivalent arrangement, so there are really only (Q-1)! / 2 possibilities.

So, we can start by saying there is 1 possible arrangement, and then, for each table, multiply that number by (P choose Q) and (Q-1)! / 2, to reflect populating that table. The value of P will diminish as we seat more and more people, and the value of Q will depend on whether we are looking at one of our "more full" or "less full" tables. It does not matter what order we process the tables in.

However, after we have done this, we still need to correct for some overcounting. For any particular arrangement, we have a huge number of duplicates. Specifically, we could have enumerated the M "more full" tables in the arrangement in any one of M! possible orders, and the L "less full" tables in any one of L! possible orders. Dividing our total by M! and L! eliminates this redundancy and yields the correct answer.

Analysis: Zathras

Zathras: Analysis

Small dataset

The Small dataset is an exercise in understanding the rules of the described system, and then turning them into code. There are some issues to watch out for:

- If we use floating-point numbers, we must avoid precision issues and ensure that we round down appropriately whenever the rules require it. It is safer to reframe the calculations to use only integers; in most languages, integer division already has the round-down property that the rules require.
- When determining the types of babies, we must see how many guaranteed babies of each type we get before we can know the size of the pool of remaining babies to divvy up. Suppose we have **α** = 40 and **β** = 20, and 4950 of each type of Zathrinian. Then we must create 99 babies; floor(99 * 0.4) = 39 of them must be acrobots, floor(99 * 0.2) = 19 of them must be bouncoids, and then the remaining 99 - 39 - 19 = 41 of them must be 20 acrobots and 21 bouncoids, for a total of 59 acrobots and 40 bouncoids. If we had instead tried to directly calculate the number of remaining babies as 100 - 40 - 20 = 40% of 99, we would have gotten 39 or 40 (depending on how we rounded), not 41.

Since **Y** ≤ 100 in the Small dataset, simulation is fast enough. Note that it is possible for **Y** to equal 0!

Large dataset

In the Large dataset, **Y** can be as large as 10^{15}; there is not enough time in the submission window to simulate that many steps. (Most contestants who attempted but did not pass this dataset got the "Time expired" verdict.) We need another insight.

The example in the problem statement provides a hint: the numbers of acrobots and bouncoids reach stable values that do not change from year to year. If we ever find that those numbers are the same two years in a row, we know they will be the same forever (since the rules do not change), and so we can stop the simulation. But how can we convince ourselves that this will necessarily happen fast enough, or at all? If there is a cycle of length greater than 1 — that is, if we see the same pair of acrobot and bouncoid population values more than once, but not back-to-back — then this strategy will fail. It turns out to be difficult to prove that there are no such cycles, or even to set bounds on how long the simulation can go on.

At this point, one option is to just assume that the simulation always reaches a stable point, and this assumption turns out to be correct. Another option is to implement a more cautious simulation that includes a cycle detection algorithm that uses constant memory; if we detect a cycle, we can then figure out where in the cycle the simulation would end, without actually running the remainder (but, again, this turns out not to happen). Since there are just over 10^{12} possible ordered pairs of acrobot and bouncoid population values, the simulation cannot possibly take more steps than that, and it is possible to convince yourself that much tighter upper bounds exist. Under the given rules, the percentage of acrobots approaches **α** + (100 - **α** - **β**) / 2, and the percentage of bouncoids approaches **β** + (100 - **α** - **β**) / 2, and these approaches are at more or less exponential speed, although they may oscillate around their final values for a while (and it is even possible for the total population size to increase, up to a

point). The behavior of the exponential approach is "smooth" — small changes to a set of initial values do not change the behavior much — so you can make yourself more confident overall by experimenting with various values. In practice, each simulation takes at most several tens of thousands of steps.